Gemini Enterprise Deployment & Rollout — Licensing, Enablement & Feature Configuration

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Outcome . . . 4

Approach . . . 4

Why it is safe and governed . . . 4

Solution Overview 5

What this is . . . 5

How it works . . . 5

Reference design . . . 6

Benefits . . . 6

Limitations & boundaries . . . 7

Deployment model . . . 7

Requirements 7

1. Gemini / AI licensing owned by the tenant . . . 7

2. Governance guardrails already in place (the separate governance solution) . . . 8

3. Super-admin (or delegated admin) with the right privileges . . . 8

4. A rollout-ready OU / group structure . . . 8

5. Admin SDK access for measurement (Reports API) . . . 9

6. A readiness/training package and adoption target . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 10

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

Implementation 12

Stage 0 — Assess (read-only) . . . 12

Stage 1 — Configure (pilot scope only) . . . 13

Stage 2 — Pilot . . . 14

Stage 3 — Validate (the gates) . . . 14

Stage 4 — Expand (wave by wave) . . . 15

Testing & Validation 15

T0 — Guardrail gate (Gate G0) . . . 15

T1 — License readiness & feasibility . . . 15

T2 — Scope correctness (Gate G1) . . . 16

T3 — Licensing correctness (Gate G2) . . . 16
T4 — Adoption threshold (Gate G3) . . . 16
T5 — Configuration snapshot & rollback rehearsal . . . 16
T6 — End-to-end pilot acceptance . . . 16
Troubleshooting 17
Users have a license but see no Gemini experience . . . 17
Gemini appears for users who should not have it yet (scope bleed) . . . 17
Gate G0 fails — a required guardrail is missing . . . 17
A capability's Admin-console toggle does not match the docs . . . 18
Adoption gate (G3) fails — usage below target . . . 18
`gemini_adoption_report.py` shows no Gemini usage metrics . . . 18
License assignment "succeeds" but seat count doesn't change . . . 18
A wave regressed after enablement and must be reversed . . . 18
Automation can't authenticate . . . 19
Best Practices 19
Rollout discipline . . . 19
Feature-surface caution . . . 20
Adoption . . . 20
Operations & safety . . . 20
Deliverables Summary 20
Documentation . . . 21
The 5 Python tools . . . 21
Appendix A — PowerShell Scripts 23

Executive Summary

Buying Gemini licenses is not a rollout. A large organization can hold thousands of Gemini Enterprise / AI seats and still see almost no use of them — because the features were never enabled for the right people, users were never made ready, and no one measured adoption. This solution closes that gap: it is the **hands-on deployment and rollout** of Gemini Enterprise to the workforce.

Outcome

Gemini Enterprise **actually deployed and adopted** — licenses assigned to the right people, features enabled and configured for the right scopes in the Google Admin console (Gemini in Workspace apps, the Gemini app, Gems, and NotebookLM Enterprise **where available**), rolled out in phases by OU/group, users trained, and adoption measured against a baseline and reported to the sponsor with data.

Approach

The rollout follows a disciplined, pilot-first model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback for every production-affecting change:

- **Assess** the license position, confirm governance guardrails are live, and baseline adoption.
- **Configure** licensing and feature enablement for a bounded pilot cohort only.
- **Pilot** with training and measurement over a fixed window.
- **Validate** against four objective gates: guardrails respected, scope correct, licensing correct, and adoption above target.
- **Expand** wave by wave, repeating enable → train → measure, with per-wave rollback if a wave regresses.

Five Python tools (Admin SDK / Reports API) make assessment, enablement planning, validation, and adoption reporting reproducible and **safe by default** — read-only or dry-run unless an apply is explicitly requested, with no credentials or keys in code.

Why it is safe and governed

- **Enablement never outruns governance.** The security guardrails for Gemini (DLP, data-region, model-usage, admin-security) are owned by a **separate governance solution** and are re-validated at a gate before every wave; this deployment turns features on only where the org is already protected.
- **Scoped and reversible.** Nothing is enabled org-wide before passing a representative pilot; each wave is scoped to named OUs/groups and can be rolled back without disturbing prior waves.
- **Measured, not assumed.** Adoption is baselined and trended per wave, so the investment is reported on objectively — and expansion is earned by clearing the adoption gate, not merely scheduled.

The result is a Gemini Enterprise capability the workforce genuinely uses — deployed at enterprise scale, in phases, safely, and with the numbers to prove it.

Solution Overview

What this is

This is the **hands-on deployment and rollout** of Gemini Enterprise to the workforce of a large organization (80k–150k+ users) that runs Google Workspace. Buying Gemini or AI add-on licenses is a procurement event, not a rollout. This solution turns that purchase into people actually using Gemini — **safely, in phases, and measurably**: license assignment, enabling and configuring the Gemini features in the Google Admin console, a phased enablement plan by organizational unit (OU) and group, user readiness and training, and adoption measurement through the Admin SDK Reports API.

The organization has (or is standing up) two **separate, complementary** capabilities that this solution deliberately does **not** re-do:

- **Gemini Enterprise Secure Adoption & Governance** (the *governance/guardrails* product) — the DLP,

data-region, model-usage, and admin-policy guardrails that make Gemini *safe* to turn on. This deployment solution **assumes those guardrails are in place** and validates that they are respected as features go live. It configures *enablement*, not the security posture.

- **Gemini Connector Strategy & Agent Development Framework** (the *connector/agent* product) — the

strategy for grounding Gemini in enterprise data sources and building custom agents/extensions. This deployment solution **references** connectors as an adoption lever but does **not** build them.

Keep the boundary crisp: **governance decides what is allowed, connectors decide what Gemini can reach, and this solution rolls the allowed, reachable capability out to real users and drives adoption.**

How it works

The work follows the house deployment model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback at every production-affecting step:

1. **Assess.** Inventory the current licensing position (which SKUs are owned, how many are assigned

vs. available), confirm the governance guardrails are live, and confirm the OU/group structure that the rollout will target. Establish an **adoption baseline** so improvement is measurable.

1. **Configure.** Assign licenses to the pilot cohort, then enable and configure the Gemini features

for the pilot scope in the Admin console — scoped to a named OU/group, not the whole org.

1. **Pilot.** Run a bounded pilot cohort (`Rollout Cohort – Pilot – Wave 0`), deliver readiness/training,

and gather usage + feedback for a fixed window.

1. **Validate.** Check the pass/fail gates: features are enabled only for the intended groups, the

guardrails are still respected, licenses are assigned correctly, and pilot adoption clears the target threshold.

1. **Expand.** Roll out wave by wave to the rest of the workforce on a documented cadence, repeating the enable → train → measure loop per wave, with rollback available if a wave regresses.

Reference design

The rollout is organized around three consistently named artifacts, reused verbatim in the implementation, testing, troubleshooting, and scripts:

- **Licensing** — a license-assignment plan mapped to OUs/groups (`License Assignment – Gemini – v1`).

Assignment is what actually consumes a paid seat; enabling a feature at the OU level does not by itself grant a per-user Gemini/AI license.

- **Feature enablement scope** — a per-OU/group enablement matrix (`Feature Enablement Matrix – Workforce – v1`) recording which Gemini capabilities are turned on for which scope. Feature names and availability evolve and some are **preview**; the matrix records the *decision* and always carries a "verify in your tenant" note next to any capability whose exact Admin-console control or availability is version-dependent.
- **Rollout waves** — an ordered set of cohorts (`Rollout Cohort – Pilot – Wave 0`, `Rollout Cohort – Wave 1`, … `Rollout Cohort – Wave N`) each mapped to OUs/groups, with an enablement date, a training package, and an adoption target.

Measurement rides on the **Admin SDK Reports API** (Gemini/Workspace usage activity and, where exposed, aggregate usage counters), read-only, to produce an adoption trend per wave against the baseline.

> ***Feature-surface caveat (read this).*** *Gemini Enterprise capabilities — *Gemini in Workspace apps* (Gmail, Docs, Sheets, Slides, Meet, Drive), the **Gemini app**, **Gems** (custom saved personas/assistants), and **NotebookLM Enterprise** — are actively evolving, and some are in **preview** or roll out per edition/region. This solution states feature *intent* and the *place in the Admin console you configure it* at a level that stays correct, and explicitly flags **"verify in your tenant"** wherever the exact toggle, menu label, availability, or Reports API metric is version-dependent. It does **not** invent exact Admin-console switch names it cannot stand behind.*

Benefits

- **A rollout, not a purchase.** Licenses are assigned to the right people, features are enabled for the right scopes, and users are trained to use them — the gap between "we bought Gemini" and "our people use Gemini" is closed.
- **Phased and reversible.** Pilot-first with pass/fail gates and per-wave rollback means a bad wave is contained, not org-wide.
- **Measured.** Adoption is baselined and tracked per wave from the Reports API, so the investment is reported on with data, not anecdotes.
- **Safe by construction.** Enablement is scoped and the governance guardrails are validated at each

gate, so speed of rollout never outruns the security posture.

• **Repeatable.** The five Python tools make assessment, enablement planning, validation, and adoption reporting reproducible and dry-run-safe.

Limitations & boundaries

• **Not governance.** DLP, data-region/residency, model-usage, and admin-security policy for Gemini are owned by the separate governance solution; here they are a validated prerequisite.

• **Not connectors/agents.** Grounding Gemini in enterprise data and building custom agents/extensions is the separate connector solution; referenced, not delivered.

• **Feature availability is tenant- and edition-specific.** Exact capabilities, preview status, and Admin-console controls must be confirmed live in the target tenant — this solution flags where.

• **Licensing/commercial terms deferred.** SKU eligibility, entitlement, and contract terms are between the organization and Google/its reseller; this solution assigns the licenses the tenant holds.

• **Change management is delivered as a rollout mechanism**, not a full organizational-change program;

a dedicated adoption/change-management engagement exists separately if deeper OCM is required.

Deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, every production-affecting change gated by an explicit pass/fail check and backed by a documented rollback. No feature is enabled org-wide before it has passed the pilot gate for a representative cohort.

Requirements

These are the **solution-specific** prerequisites for deploying and rolling out Gemini Enterprise. They build **on top of** the shared Google-category requirements in `global-requirements-google.md` — that file already covers the Workspace/Cloud Identity tenant and super-admin, the identity source of truth and provisioning, DNS control, a pilot OU/group and change control, and the security/compliance stakeholders. Verify the shared six first; then verify each of the following before starting the implementation steps.

> *Where a capability below is version- or preview-dependent (a Gemini feature, an Admin-console control path, or a Reports API metric),* ***verify it live in your tenant*** *— do not assume the label or availability from this document.*

1. Gemini / AI licensing owned by the tenant

• **Description:** The tenant holds the Gemini Enterprise / AI add-on entitlement (as a Workspace add-on SKU or the applicable Gemini edition) with enough **available seats** for the planned waves.

• **Why required:** Feature enablement at the OU level does not grant a per-user paid capability;

license assignment is what consumes a seat and unlocks the user experience. Rollout waves are bounded by available seats.

- **How to verify:** In **admin.google.com** → **Billing** → **Subscriptions**, confirm the Gemini/AI subscription is present and note **licenses assigned vs. total**. Cross-check with the assessment script (`gemini_license_readiness.py`). Exact SKU name/edition is entitlement-dependent — **verify in your tenant**.
- **How to implement if missing:** Procure/adjust the subscription through Google or an authorized reseller before rollout; do not start a wave you cannot license end to end.

2. Governance guardrails already in place (the separate governance solution)

- **Description:** The Gemini **security/data guardrails** — DLP posture, data-region/residency settings, model-usage/admin-security policy, and any regulated-workload controls — are configured and validated by the separate **Gemini Enterprise Secure Adoption & Governance** solution.
- **Why required:** This deployment turns features **on**; it must never do so ahead of the guardrails that make them safe. Every rollout gate re-validates that guardrails are respected.
- **How to verify:** Confirm with the governance owner that the guardrail baseline is live for the scopes you will enable; run `gemini_guardrail_precheck.py` (read-only) against the target OUs.
- **How to implement if missing:** Complete/deploy the governance solution for the target scopes **before** enabling Gemini features there. Do not proceed on an ungoverned scope.

3. Super-admin (or delegated admin) with the right privileges

- **Description:** An administrator with the Google Admin **privileges** to manage the Gemini/AI service, assign licenses, and manage the target OUs/groups — ideally a **delegated admin role** scoped to those tasks rather than a full super-admin for day-to-day rollout work.
- **Why required:** License assignment and per-OU service configuration require these privileges; least-privilege delegation reduces blast radius during a large rollout.
- **How to verify:** In **Admin console** → **Account** → **Admin roles**, confirm the operator's role grants license management and service (Gemini) settings for the target org tree.
- **How to implement if missing:** Create a delegated admin role with the minimum privileges for licensing + Gemini service management and assign it to the rollout operators.

4. A rollout-ready OU / group structure

- **Description:** An OU tree and/or Google Groups that cleanly express the **rollout waves** — a pilot cohort plus the ordered production waves — so features and licenses can be scoped per wave without catching unintended users.

• **Why required:** Gemini service settings and license assignment are applied by OU/group; clean scoping is what makes the rollout phased and reversible.

• **How to verify:** Confirm the pilot OU/group from the shared requirements exists, and that additional wave groups exist or can be created; run `gemini_rollout_wave_planner.py --dry-run` to map users to waves against the structure.

• **How to implement if missing:** Create the wave OUs/groups (`Rollout Cohort – Pilot – Wave 0`, `Rollout Cohort – Wave 1`, …) and confirm membership is driven from the identity source of truth.

5. Admin SDK access for measurement (Reports API)

• **Description:** The **Admin SDK** with the **Reports API** enabled and an authorized principal (delegated admin credentials via Application Default Credentials, or a properly authorized service account with domain-wide delegation) able to read usage/activity reports **read-only**.

• **Why required:** Adoption measurement and license/usage reporting are driven from the Reports API; without it there is no baseline and no per-wave trend.

• **How to verify:** Confirm the Admin SDK is enabled for the project and that the reporting principal can call the Reports API; run `gemini_adoption_report.py --dry-run`. The **exact Gemini usage metrics/parameters exposed** by the Reports API are version-dependent — **verify in your tenant** which activity events and usage counters are available.

• **How to implement if missing:** Enable the Admin SDK API, grant the reporting principal the read-only reports scope, and (if using a service account) authorize domain-wide delegation for the reports read scope only.

6. A readiness/training package and adoption target

• **Description:** A per-wave **enablement/training package** (quick-start guidance, example prompts, Gems/NotebookLM usage where available, and role-specific use cases) and an agreed **adoption target** (the pass threshold for a wave's validation gate).

• **Why required:** Turning a feature on without readiness produces licenses that are assigned but unused; the adoption target is the objective pass/fail criterion for expanding.

• **How to verify:** Confirm the training package exists for the pilot and an adoption threshold is agreed with the sponsor (e.g. a target active-user percentage within the wave window).

• **How to implement if missing:** Build the readiness package for the pilot and agree the adoption target with the sponsor before enabling the pilot cohort.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Follow these steps in order. They implement the house model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for every production-affecting change. Every artifact is named on first creation with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, troubleshooting, and the scripts.

> ***Console-path discipline.*** *Google Admin console menus and Gemini feature labels change over time, and some capabilities are* ***preview*** *or edition/region-dependent. Where a step names a control, the *navigation intent* is stable; if the exact label or availability differs in your tenant,* ***verify in your tenant*** *and record the actual path in the* `Feature Enablement Matrix – Workforce – v1`*. Do not enable a capability whose behavior you have not confirmed.*

Stage 0 — Assess (read-only)

0.1 Confirm licensing position

1. Sign in to **admin.google.com** as a super-admin (or the delegated rollout admin).
2. Go to **Billing** → **Subscriptions** and record the Gemini/AI subscription, **total seats**, and **seats assigned vs. available**.
1. Run the assessment tool to capture this machine-readably and flag shortfalls:

```
python gemini_license_readiness.py --report license-baseline.json
```

This is **read-only**. It records the baseline into `License Assignment – Gemini – v1`.

0.2 Confirm governance guardrails are live

1. With the governance owner, confirm the **Gemini Enterprise Secure Adoption & Governance** baseline (DLP, data-region, model-usage, admin-security policy) is deployed for the OUs you will enable.
1. Run the read-only pre-check:

```
python gemini_guardrail_precheck.py --ou "/Pilot/Wave0" --dry-run
```

1. **Gate G0 (guardrail gate):** if any target scope is ungoverned, **stop** — do not enable Gemini there until governance is complete. This gate is re-checked before every wave.

0.3 Establish the adoption baseline

1. Confirm the Admin SDK Reports API is reachable for the reporting principal.
2. Capture the pre-rollout usage baseline:

```
python gemini_adoption_report.py --baseline --out adoption-baseline.json
```

Record which Gemini usage metrics the Reports API actually exposes in your tenant (**verify in your tenant** — metric availability is version-dependent). The baseline anchors every later trend.

0.4 Plan the waves

1. Map users to rollout waves against the OU/group structure:

```
python gemini_rollout_wave_planner.py --plan waves.yaml --dry-run --out wave-plan.json
```

1. This produces the ordered cohorts — `Rollout Cohort – Pilot – Wave 0`, `Rollout Cohort – Wave 1`,

… `Rollout Cohort – Wave N` — each with target OUs/groups, an enablement date, a training package, an adoption target, and the seat count required (checked against the license baseline).

Assess exit criteria: license baseline captured, Gate G0 passed for the pilot scope, adoption baseline recorded, wave plan produced and seat-feasible.

Stage 1 — Configure (pilot scope only)

1.1 Assign licenses to the pilot cohort

1. Assign Gemini/AI licenses to `Rollout Cohort – Pilot – Wave 0`. In the Admin console this is done

via **Billing** → **Subscriptions** → **(Gemini/AI subscription)** → **Assign licenses**, scoped to the pilot OU/group; automatic licensing by OU may be used where the tenant supports it — **verify in your tenant**.

1. Or apply the planned assignment with the helper (dry-run first, then apply):

```
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --assign-licenses --dry-run
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --assign-licenses --apply
```

1. Record assignments in `License Assignment – Gemini – v1`.

1.2 Enable and configure Gemini features for the pilot

Configure each capability **scoped to the pilot OU/group only**, and record every decision (including the exact control path you used and any preview flag) in `Feature Enablement Matrix – Workforce – v1`:

1. **Gemini in Workspace apps** — enable the Gemini side-panel / assistive features in Gmail, Docs,

Sheets, Slides, Meet, and Drive for the pilot OU. In the Admin console these live under the **Generative AI / Gemini** service settings and/or the individual app service settings; the exact grouping is version-dependent — **verify in your tenant**.

1. **The Gemini app** — enable access to the standalone Gemini app/experience for the pilot OU where

your edition provides it.

1. **Gems** (custom saved assistants/personas) — enable and, if desired, seed a small set of

org-relevant starter Gems for the pilot. Availability and the admin control for Gems are evolving — **verify in your tenant** whether it is generally available or preview for your edition.

1. **NotebookLM Enterprise** — enable **only where available** for your edition/region. This is

frequently **preview or edition-gated**; if it is not clearly available and governed for your tenant, mark it *Deferred* in the matrix and **do not enable it** in the pilot. **Verify in your tenant.**

1. For each capability enabled, note whether it is **GA or preview** and attach the "verify in your

tenant" flag in the matrix where the control path or availability was not certain.

1.3 Prepare readiness for the pilot

1. Attach the pilot training package to `Rollout Cohort – Pilot – Wave 0` (quick-start, example prompts, Gems/NotebookLM usage where enabled, role-specific use cases).
1. Snapshot the intended configuration for validation and rollback:

```
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --snapshot --out pilot-config-snapshot.json
```

Rollback (Stage 1): to revert the pilot, disable the enabled features for the pilot OU/group and unassign the licenses:

```
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --revert --from pilot-config-snapshot.json --dry-r
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --revert --from pilot-config-snapshot.json --apply
```

Feature settings and license assignment revert independently — reverting a setting does not release a seat; unassign licenses explicitly to free seats.

Stage 2 — Pilot

1. Announce enablement to `Rollout Cohort – Pilot – Wave 0` and deliver the training package.
2. Run the pilot for the fixed window agreed with the sponsor.
3. Collect usage over the window:

```
python gemini_adoption_report.py --cohort "Pilot-Wave0" --since <pilot-start> --out pilot-adoption.json
```

1. Collect qualitative feedback (blockers, missing use cases, connector gaps to hand to the separate connector solution).

Stage 3 — Validate (the gates)

Run the consolidated validation before expanding. All gates must pass; see `05-testing.md` for exact expected results.

- **Gate G1 — scope correctness:** features are enabled **only** for the intended OUs/groups (no bleed to unintended users).

```
python gemini_feature_enablement.py --ou "/Pilot/Wave0" --verify-scope
```

- **Gate G2 — licensing correctness:** assigned seats match the cohort; no over- or under-assignment.

```
python gemini_license_readiness.py --verify-cohort "Pilot-Wave0"
```

- **Gate G0 (re-check) — guardrails respected:** governance guardrails still hold for the enabled scope.

```
python gemini_guardrail_precheck.py --ou "/Pilot/Wave0"
```

- **Gate G3 — adoption threshold:** pilot adoption clears the agreed target vs. baseline.

```
python gemini_adoption_report.py --cohort "Pilot-Wave0" --against adoption-baseline.json --gate <target>
```

Validate exit criteria: G0, G1, G2, G3 all pass. If any fails, remediate or roll back the pilot (Stage 1 rollback) before proceeding — do **not** expand on a failed gate.

Stage 4 — Expand (wave by wave)

For each production wave `Rollout Cohort – Wave N` in the wave plan, repeat the loop:

1. **Guardrail pre-check (G0):** `python gemini_guardrail_precheck.py --ou "<wave-ou>"` — must pass

before enabling.

1. **Assign licenses** for the wave (dry-run then apply), checking seat availability against

`License Assignment – Gemini – v1`.

1. **Enable features** for the wave, matching the validated pilot configuration recorded in

`Feature Enablement Matrix – Workforce – v1` (dry-run then apply; snapshot for rollback first).

1. **Train** the wave with the readiness package.

2. **Measure** with `gemini_adoption_report.py --cohort "Wave-N"` and run the same G1/G2/G3 gates for

the wave.

1. **Rollback (per wave):** identical to the Stage 1 rollback, scoped to the wave OU/group, using that

wave's config snapshot. A regressing wave is reverted without touching prior waves.

Proceed to the next wave only after the current wave passes its gates. Continue until the full workforce is enabled, trained, and measured, and the adoption trend is reported to the sponsor against the baseline.

Testing & Validation

Each procedure below has an explicit **expected pass/fail**. These are the gates referenced in `03-implementation.md`. Run them for the pilot cohort first, then for every production wave before it is allowed to expand. All tools are read-only in their verify/gate modes.

> *Where a check depends on a Gemini feature label or a Reports API metric, the **presence and name of that signal is version-dependent** — confirm the actual signal exists in your tenant and record it in `Feature Enablement Matrix – Workforce – v1`. A missing metric is a "verify in your tenant" finding, not automatically a fail.*

T0 — Guardrail gate (Gate G0)

Purpose: confirm the governance guardrails are live for the scope before/after enablement. **Run:** `python gemini_guardrail_precheck.py --ou "<wave-ou>"` **Pass:** the tool reports the expected DLP/data-region/model-usage/admin-security baseline present for the scope, with no ungoverned gap. **Fail:** any target OU/group is ungoverned, or a guardrail expected by the governance baseline is absent → **stop**; do not enable (or, if already enabled, roll back) this scope until governance is restored.

T1 — License readiness & feasibility

Purpose: confirm enough seats exist and the baseline is captured. **Run:** `python gemini_license_readiness.py --report license-baseline.json` **Pass:** available seats ≥ the

seats required by the remaining waves; baseline written. **Fail:** projected seat shortfall for a planned wave → resolve licensing before starting that wave.

T2 — Scope correctness (Gate G1)

Purpose: features are enabled **only** for the intended OUs/groups. **Run:** `python gemini_feature_enablement.py --ou "<wave-ou>" --verify-scope` **Pass:** every enabled capability is applied to exactly the intended OU/group; no unintended users inherit the setting; the observed configuration matches `Feature Enablement Matrix – Workforce – v1`. **Fail:** any capability is enabled for a broader scope than intended, or the live config diverges from the matrix → correct the scope (or roll back) before proceeding.

T3 — Licensing correctness (Gate G2)

Purpose: assigned seats match the cohort exactly. **Run:** `python gemini_license_readiness.py --verify-cohort "<cohort>"` **Pass:** the set of users with an assigned Gemini/AI license equals the cohort membership — no over-assignment (wasted seats) and no under-assignment (users who will see no Gemini experience). **Fail:** a diff between assigned licenses and cohort membership → reconcile assignment before the gate passes.

T4 — Adoption threshold (Gate G3)

Purpose: the cohort's adoption clears the agreed target relative to the baseline. **Run:** `python gemini_adoption_report.py --cohort "<cohort>" --against adoption-baseline.json --gate <target>` **Pass:** measured active-usage for the cohort within the window meets or exceeds the agreed adoption target over the baseline; the tool prints `GATE: PASS`. **Fail:** adoption below target → do not expand; investigate readiness/training, connector gaps, or a feature-availability issue, remediate, and re-measure. Persisting failure after remediation is a signal to pause the wave.

T5 — Configuration snapshot & rollback rehearsal

Purpose: confirm the rollback path works before it is needed. **Run (dry-run):**

```
python gemini_feature_enablement.py --ou "<wave-ou>" --snapshot --out <wave>-config-snapshot.json
python gemini_feature_enablement.py --ou "<wave-ou>" --revert --from <wave>-config-snapshot.json --dry-run
```

Pass: the snapshot captures the enabled capabilities and license assignment for the scope, and the dry-run revert reports the exact set of changes it *would* make to return the scope to pre-wave state. **Fail:** snapshot incomplete or dry-run revert cannot reconstruct the pre-wave state → fix before enabling the wave (never enable a scope you cannot cleanly revert).

T6 — End-to-end pilot acceptance

Purpose: the pilot is a true dress rehearsal for expansion. **Method:** with the pilot cohort enabled, trained, and measured, confirm **all** of G0, G1, G2, G3 pass together and a sample of pilot users can perform

representative tasks (Gemini in an app, the Gemini app, a Gem, and NotebookLM Enterprise **where enabled**). **Pass:** all gates green and representative tasks succeed for the pilot cohort → cleared to expand. **Fail:** any gate red or representative task blocked → remediate within the pilot before Wave 1; expansion is blocked on a green pilot.

Troubleshooting

Common failure modes during Gemini Enterprise deployment and rollout, and how to resolve them. Each maps back to the artifacts and gates in `03-implementation.md` and `05-testing.md`.

> *Many symptoms trace to a **feature label, availability, or Reports metric that differs in your tenant**. When a step's control does not match, that is a **"verify in your tenant"** finding — confirm the live path and record it in `Feature Enablement Matrix – Workforce – v1` before treating it as a defect.*

Users have a license but see no Gemini experience

- **Likely cause:** the feature is enabled at the OU level but the user's **license is not assigned**,

or the user is in an OU where the capability was not enabled.

- **Resolve:** run `gemini_license_readiness.py --verify-cohort "<cohort>"` (Gate G2). Reconcile any

under-assignment against `License Assignment – Gemini – v1`. Confirm the user's OU is in the enablement scope in `Feature Enablement Matrix – Workforce – v1`. Note that feature settings and license assignment are independent — one being correct does not imply the other.

Gemini appears for users who should not have it yet (scope bleed)

- **Likely cause:** a capability was enabled on a **parent OU** and inherited by child OUs, or on a

group broader than the intended cohort.

- **Resolve:** run `gemini_feature_enablement.py --ou "<wave-ou>" --verify-scope` (Gate G1). Re-scope

the setting to exactly the intended OU/group. If needed, roll the scope back from its snapshot (`--revert --from <snapshot>`) and re-apply correctly. Confirm the live config matches the matrix.

Gate G0 fails — a required guardrail is missing

- **Likely cause:** the scope is **ungoverned** — the separate governance solution's baseline (DLP,

data-region, model-usage, admin-security) is not deployed for this OU/group.

- **Resolve: stop** — do not enable Gemini on this scope. Engage the governance owner to deploy the

baseline, then re-run `gemini_guardrail_precheck.py --ou "<wave-ou>" --expect guardrail-baseline.json`. If the check returns INDETERMINATE, the guardrail state is UNKNOWN, not absent — verify with the governance owner and supply a complete expected baseline; never treat UNKNOWN as pass.

A capability's Admin-console toggle does not match the docs

- **Likely cause:** Gemini feature grouping and labels **evolve**; the capability may be under a

different service, renamed, or in **preview** for your edition/region.

- **Resolve: verify in your tenant** — locate the actual control, record the exact path and whether

it is GA or preview in the matrix, and only then enable. For NotebookLM Enterprise and other preview/edition-gated capabilities, if it is not clearly available **and** governed, mark it *Deferred* and do not enable it.

Adoption gate (G3) fails — usage below target

- **Likely cause:** users are licensed and enabled but not using Gemini — commonly weak readiness/

training, unclear role-specific value, or **connector gaps** (Gemini can't reach the data users need, which is the separate connector solution's domain).

- **Resolve:** do not expand. Investigate: confirm training was delivered for the wave; strengthen

role-based use cases and starter Gems; capture connector gaps as feedback to the connector solution. Re-measure with `gemini_adoption_report.py --cohort "<cohort>" --against adoption-baseline.json --gate <target>`. Persistent failure after remediation is a signal to pause the wave, not to lower the bar.

`gemini_adoption_report.py` shows no Gemini usage metrics

- **Likely cause:** the Reports API **parameter names for Gemini usage are version-dependent** and may

live under app-specific reports rather than customer-level counters; or the reporting principal lacks the read scope; or data simply hasn't landed (Reports data typically lags ~2 days).

- **Resolve:** confirm the Admin SDK Reports API is enabled and the principal holds the read-only

reports scope. Query a date at least 2 days back. **Verify in your tenant** which application/ parameter carries Gemini activity and pass it via `--active-metric`. The tool reports INDETERMINATE rather than inventing a metric — treat that as "identify the real metric", not "pass".

License assignment "succeeds" but seat count doesn't change

- **Likely cause:** assignment vs. the subscription's total/available seats is read in two different

places — the Licensing API reports **assignments**, while total/available seats are shown under **Billing** → **Subscriptions** and are not exposed the same way.

- **Resolve:** re-check total/available seats in **Billing** → **Subscriptions** (verify in your tenant),

and reconcile assignments with `gemini_license_readiness.py`. If waves are seat-constrained, re-run `gemini_rollout_wave_planner.py --license-baseline license-baseline.json` to confirm feasibility before assigning more.

A wave regressed after enablement and must be reversed

- **Likely cause:** a wave introduced a scope, guardrail, or adoption problem that can't be fixed

in place quickly.

- **Resolve:** use the **per-wave rollback**: `gemini_feature_enablement.py --ou "<wave-ou>" --revert

--from <wave>-config-snapshot.json (dry-run, then --apply`), and unassign that wave's licenses explicitly (reverting a feature does not release seats). Prior waves are untouched. Fix the root cause, then re-enable the wave through the normal gated loop.

Automation can't authenticate

- **Likely cause:** no Application Default Credentials present, wrong/missing API scopes, or (for a

service account) domain-wide delegation not authorized for the required read scope.

- **Resolve:** establish ADC (`gcloud auth application-default login`) or configure an authorized

service account **from the environment** — never embed a key in code. Grant exactly the read scopes the tool needs (licensing/directory read for readiness; reports read for adoption). If a client library is not installed, the tools print the equivalent Admin-console steps so the run is still actionable; install with `pip install google-api-python-client google-auth` to automate.

Best Practices

Durable practices for deploying and driving adoption of Gemini Enterprise across a large workforce.

Rollout discipline

- **License assignment is the real switch.** Enabling a feature at the OU level does not grant a paid

seat; assigning the Gemini/AI license does. Always verify assignment (`License Assignment – Gemini – v1`) matches the cohort — under-assignment leaves trained users with no Gemini experience; over-assignment wastes seats.

- **Never enable ahead of governance.** Gate G0 (guardrail pre-check) runs before every wave, not just

the pilot. Speed of rollout must never outrun the security posture owned by the separate governance solution.

- **Pilot as a true dress rehearsal.** The pilot cohort exercises every capability, the training

package, the measurement pipeline, and the rollback — so Wave 1 is a repeat, not a first attempt.

- **One config of record.** `Feature Enablement Matrix – Workforce – v1` is the single source of truth

for what is enabled where, whether each item is GA or preview, and the exact Admin-console control used. Every wave matches the validated pilot configuration; drift is a Gate G1 failure.

- **Snapshot before every wave.** Take a config snapshot before enabling a wave so rollback is

mechanical and scoped — a regressing wave is reverted without disturbing prior waves.

Feature-surface caution

- **Treat feature names and toggles as tenant-specific.** Gemini capabilities evolve and some are **preview** or edition/region-gated. Record the *actual* control path you used per capability and carry a "verify in your tenant" note wherever availability or the label was not certain.
- **Don't enable what you can't govern and verify.** For preview/edition-gated capabilities (frequently NotebookLM Enterprise, and evolving controls like Gems), mark them *Deferred* until they are clearly available **and** covered by the governance baseline — never enable on assumption.
- **Separate the three concerns.** Governance decides *allowed*, connectors decide *reachable*, this deployment rolls out the *allowed + reachable* capability. Keep connector/agent gaps as feedback to the connector solution rather than solving them here.

Adoption

- **Baseline first, then trend.** Capture the pre-rollout adoption baseline before any wave; every wave is measured *against* it, so the investment is reported with data, not anecdotes.
- **Train at the moment of enablement.** Deliver the readiness package as the wave goes live — a seat assigned without readiness becomes an unused seat.
- **Make adoption the gate.** Gate G3 (adoption threshold) is an objective pass/fail; a wave below target is investigated (readiness, connector gaps, feature availability) and remediated before expanding — expansion is earned, not scheduled.
- **Role-based use cases beat generic demos.** Package example prompts, starter Gems, and (where enabled) NotebookLM use cases per role; concrete "here's how your job gets easier" material drives adoption more than feature tours.

Operations & safety

- **Least-privilege admin.** Run day-to-day rollout with a delegated admin role scoped to licensing + Gemini service management, not a full super-admin.
- **Dry-run everything that writes.** The configuration/assignment helper defaults to dry-run; apply only after reviewing the printed plan and confirming the control path in the tenant.
- **No secrets in tooling.** Automation uses ADC / `gcloud` auth or an authorized service account from the environment; no exported keys, no credentials in code or config committed to the repo.
- **Read-only measurement.** Reporting uses the Reports API with a read-only scope and (for service accounts) domain-wide delegation limited to the reports read scope only.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

• **Solution Overview** — what Gemini Enterprise deployment delivers, the licensing/feature surface,

the reference design (Licensing / Feature Enablement Matrix / Rollout Waves), the phased model, and how this deployment relates to the separate **governance** and **connector** solutions.

• **Requirements Guide** — the solution-specific prerequisites (Gemini/AI licensing owned, governance

guardrails in place, delegated admin, rollout-ready OU/group structure, Admin SDK Reports access, readiness package + adoption target), each with how to *verify* and *implement if missing*, building on the shared Google-category requirements.

• **Implementation Guide** — exact ordered steps: Assess → Configure → Pilot → Validate → Expand;

license assignment, feature enablement by OU/group, pilot, the pass/fail gates (G0–G3), and a documented per-wave rollback. Preview features flagged "verify in your tenant".

• **Testing & Validation** — the gates as runnable procedures, each with expected pass/fail (guardrail

gate, license readiness, scope correctness, licensing correctness, adoption threshold, rollback rehearsal, and end-to-end pilot acceptance).

• **Best Practices** — durable practices for licensing discipline, feature-surface caution, adoption,

and operational safety.

• **Executive Summary** — the outcome, approach, and why the rollout is safe and governed.

The 5 Python tools

All are **read-only or dry-run by default**, take no hardcoded credentials/keys (ADC / `gcloud` auth or an authorized service account from the environment), and never invent Admin-console toggles or API endpoints for version-dependent Gemini features.

1. `gemini_license_readiness.py` — *assessment (read-only).* Captures the Gemini/AI license

baseline (assigned vs. available) and verifies that a cohort's assigned licenses match its membership (Gate G2). Writes `License Assignment – Gemini – v1`.

1. `gemini_feature_enablement.py` — *configuration helper (dry-run default).* Plans/applies license

assignment and enables/disables Gemini capabilities by OU/group, snapshots config for rollback, verifies scope correctness (Gate G1), and reverts a scope from a snapshot. Records into `Feature Enablement Matrix – Workforce – v1`. Does not call unverified feature-toggle endpoints.

1. `gemini_rollout_wave_planner.py` — *scope/config planner (read-only).* Turns a wave definition

into ordered, seat-feasible cohorts (`Rollout Cohort – Pilot – Wave 0` … `Wave N`) checked against the license baseline, with enablement dates, training packages, and adoption targets.

1. `gemini_adoption_report.py` — *validation/report generator (read-only).* Captures the adoption

baseline and per-cohort usage from the Admin SDK Reports API and evaluates the adoption gate (Gate G3) against a target relative to baseline. Reports available metrics; never fabricates a Gemini counter that the API does not expose.

1. `gemini_guardrail_precheck.py` — *policy/guardrail check (read-only).* Gate G0: confirms the governance baseline (DLP, data-region, model-usage, admin-security) is present for a scope before Gemini is enabled there. Asserts presence only; never creates or modifies a guardrail.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

gemini_adoption_report.py

gemini_feature_enablement.py

gemini_guardrail_precheck.py

gemini_license_readiness.py

gemini_rollout_wave_planner.py